

ConverterHub — Free Word ↔ PDF Converter

A privacy-first, SEO-optimized file conversion web app built with Flask + LibreOffice. Converts Word documents to PDF and PDFs to editable Word documents — no signup, no watermarks, files auto-deleted after 10 minutes.

Tech Stack

Layer	Technology
Runtime	Python 3.11
Web Framework	Flask
Conversion Engine	LibreOffice (headless)
Task Queue	Celery + Redis
Database	SQLite (dev) / PostgreSQL (prod)
Hosting	Render.com (free tier)

Project Structure

```
fileconverter/
├── app/
│   ├── __init__.py      # Flask app factory
│   ├── config.py        # Dev/Prod/Test configs
│   └── routes/
│       ├── main.py      # Homepage, sitemap, robots.txt, health
│       ├── converter.py  # Result / error pages
│       ├── blog.py       # Blog listing + post routes
│       └── api.py        # REST API: convert, status, download, delete
│   ├── services/
│       ├── converter.py  # LibreOffice conversion logic
│       ├── file_handler.py # Temp storage, cleanup
│       └── tasks.py      # Celery async tasks
│   ├── models/
│       ├── conversion.py # ConversionJob model
│       └── usage_stat.py  # DailyStat model
│   └── utils/
│       ├── validators.py # File type/MIME validation
│       ├── sanitizer.py  # Filename sanitization
│       └── seo.py        # Sitemap generator, meta tags
├── templates/           # Jinja2 HTML templates
├── static/              # CSS, JS, images
├── run.py               # App entry point
├── requirements.txt
├── Dockerfile           # Includes LibreOffice install
└── render.yaml          # Render.com deploy config
```

Local Development

Prerequisites

- Python 3.11+

- LibreOffice: `sudo apt-get install -y libreoffice` (Ubuntu/Debian)
- Redis (optional for async): `sudo apt-get install redis-server`

Setup

```
# Clone and enter project
cd fileconverter

# Create virtual environment
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Configure environment
cp .env.example .env
# Edit .env with your values

# Run the app
python run.py
```

Visit: <http://localhost:5000>

With Celery (async conversion)

```
# Terminal 1: Redis
redis-server

# Terminal 2: Celery worker
celery -A app.services.tasks.celery worker --loglevel=info

# Terminal 3: Flask
python run.py
```

API Endpoints

Method	Endpoint	Description
POST	/api/convert	Upload file, start conversion → returns <code>job_id</code>
GET	/api/status/<job_id>	Poll status: pending/processing/done/failed
GET	/api/download/<job_id>	Download converted file
DELETE	/api/file/<job_id>	Manually delete job files
GET	/sitemap.xml	Auto-generated XML sitemap
GET	/robots.txt	Crawl rules
GET	/health	Health check for Render.com

Deploy to Render.com

1. Push to GitHub
2. Create new Render account at render.com
3. "New Blueprint" → select your repo → it reads `render.yaml`
4. Update `SITE_URL` in `render.yaml` to your `.onrender.com` domain
5. Deploy — LibreOffice installs automatically via Dockerfile

Privacy & Security

- Files deleted after **10 minutes** via Celery cleanup task
- IP addresses **SHA-256 hashed** before storage — no plain IPs kept
- No user accounts, no cookies required
- Filename sanitization via `werkzeug.utils.secure_filename`
- Rate limiting: 10 conversions/hour per IP, max 2 concurrent jobs
- Max file size: 20 MB (enforced client-side + server-side)

SEO

- Unique title + meta description per page
- JSON-LD Schema markup (SoftwareApplication) on tool pages
- Auto-generated `sitemap.xml`
- `robots.txt` disallows `/api/` and `/tmp/`
- Open Graph tags for social sharing
- Mobile-first responsive design
- H1 with exact-match keywords on every page

License

MIT